Knowing Before Saying: Internal Correctness Signals in Mathematical Reasoning

Anonymous Author(s)
Affiliation
Address
`email`

Abstract

When a language model is confidently wrong, is the error undetected or left unsaid? We look inside the model to find out. Across four open models and four domains (MMLU-Pro, MATH, GPQA-Diamond, and a geometry task graded by a compiler), a linear probe on the model's hidden state predicts whether an answer is correct better than the model's own self-report in 12 of 16 cells, by a mean of +0.09 AUROC (area under the ROC curve). The gap is widest on MATH (+0.20). The probe is not just spotting hard questions: given the same question several times, it still separates the attempts that succeeded from the ones that failed. On Mistral, the signal is there before we ask. At the last token of the answer, with no confidence question yet, correctness on MMLU-Pro is readable at 0.69, against 0.57 from the question alone. A probe fitted after the attempt reads at chance before it. On Mistral, though not on GLM, self-report depends on this signal. Erase the direction after the answer is written and self-report can no longer tell right answers from wrong (0.84 $\rightarrow$ 0.56). Erase a random direction of the same size and nothing changes. Amplify it and confidence on wrong answers drops while calibration error halves. These models carry more about their own errors than they self-report, and on one model the self-report depends on it.

1 Introduction

A language model that is confidently wrong is a problem. Its stated confidence, which we call its self-report, is a signal for deciding whether to trust its answer. This paper asks whether the model's activations contain a signal of when it is wrong.

Two things could be going on. Upon giving an incorrect answer and saying it has high confidence, either the model genuinely has no idea it might be wrong and is just blindly confident, or it deep down holds the error signal in its activations but does not surface it in its self-report. Hence it is either a capability problem or a reporting problem.

In this paper we aim to tell these two apart. We read the model's internal state directly with a linear detector, a *probe*, and compare what the probe finds to the model's reported confidence. We say a fact is *represented* when a probe can read it from the model's state, and *reported* when it reaches the confidence number the model writes. The title's "knowing" is shorthand for a representation that is present, specific to the attempt, and, in at least one model, used by the report. We make no claim about awareness, and a probe alone could not support one.

Prior work has shown that a model's output probabilities are often better calibrated than its words [4], and that the truth of a given statement can be read from activations [1, 2, 5]. We apply those tools to the model's own attempts, and then intervene on the representation [6] to ask whether the report depends on it.

2 Reading the state

Models and tasks. We test four open models of three architectures: Mistral-Small-24B (dense), Qwen3.6-27B (gated linear attention interleaved with full attention), and GLM-4.7-Flash and Gemma-4-26B (mixture of experts). Each answers 150 questions, five times each, in four domains: MMLU-Pro, MATH, GPQA-Diamond, and GeoGenBench, a geometry task in which the model writes a construction and a compiler checks it against a formal specification, so its labels need no judge; in a pre-registered check of 200 attempts by two trained raters, the compiler never failed a correct construction. That is 16 cells of about 750 attempts each.

Prompting structure.

1. The model is shown the question and asked, before answering, how confident it is, from 0 to 100, that it will get it right.
2. It answers.
3. It is asked how confident it is that the answer it just gave is correct.

The probe. At the moment the model is about to write its confidence digit, we record the number and the residual-stream hidden state at that token (2,048 to 5,120 dimensions, depending on the model). The probe is a linear classifier on that vector: features are standardized, reduced to 50 principal components, and fed to an L2-regularized logistic regression that predicts whether the attempt was correct. We fit it with five-fold cross-validation grouped by question, so all five attempts at a question fall in the same fold and the probe is always scored on questions it never saw. The read depth is fixed in advance at 70% of the network (layer 28 of 40 for Mistral, 33 of 47 for GLM, 21 of 30 for Gemma-4, 45 of 64 for Qwen3.6). Every readout is scored with AUROC.

The activation state carries more than self-report. The probe beats the model's self-report in **12 of 16** cells, with one tie. The mean gain is +0.09 AUROC (Appendix D). The gap is largest on MATH, at +0.20. Mistral on MATH is a good example of what this looks like. The model is never told whether it got the answer right. Yet after attempting, it raises its confidence, and it raises it more on the attempts it got wrong (+22 points) than on the ones it got right (+16 points). The probe on the same records reads 0.83. The state tracks correctness better than the self-report does.

Why MATH. On MATH, right and wrong answers have the same shape. Both are a chain of steps ending in a boxed result, so the model cannot tell from the look of its answer whether it went wrong. Appendix A follows one attempt where a single wrong subtraction leads to a wrong answer, and the model then raises its confidence. Shape still carries a little signal. A baseline classifier that sees only surface features of the answer text, such as its length and line count, reaches 0.74, but the probe reaches 0.83. Geometry is the opposite case. A failed construction usually does not compile, so the failure is visible in the answer itself, and there the surface baseline does as well as the probe. The internal signal helps most where the failure is invisible from outside.

Controls. Three things could make a probe look better than it is; the next section tests each. It could be reading how hard the question is rather than the attempt, so we compare attempts within one question. It could be reading the shape of the output, so we compare it with the surface baseline in every cell; on Mistral, adding the probe to the surface features raises AUROC by +0.21 on MMLU-Pro, +0.07 on MATH, +0.05 on GPQA, and −0.11 on geometry. And it could be reading something that was there before the attempt, so we check the input layer, where the token is identical in every record and the probe sits at exactly 0.5. These controls were run on Mistral only, for time. Its four cells reproduce the headline.

Table 1: Mistral. Rows: training domain; columns: test domain; bold: in-domain.

train ↓ test →	**GeoGen**	**MMLU-Pro**	**MATH**	**GPQA**
GeoGen	**0.68**	0.67	0.78	0.56
MMLU-Pro	0.70	**0.75**	0.84	0.59
MATH	0.80	0.72	**0.83**	0.57
GPQA	0.60	0.69	0.74	**0.57**

3 What the signal is

Difficulty control. A probe that beats self-report could be doing something simpler. It could be learning which questions are hard. Hard questions fail more often, so a difficulty detector would predict correctness without reading the attempt. We run two checks.

First, hold the question fixed. Each question is attempted five times, so within one question, difficulty cannot vary. The probe still separates the successful attempts from the failed ones (0.72 to 0.74 in the cells of Table 3). The model's own P(True), its probability of answering "True" when asked whether it was right, drops to near chance (0.47 to 0.59).

Second, test where the probe's direction lives. The probe is a direction in the hidden state, and we can ask when that direction appears. Before the attempt, at the last token of the question, the model's state is the same across all five attempts. It can encode difficulty and nothing else. A probe trained after the attempt and read at that point scores 0.49 on MMLU-Pro, which is chance, against 0.75 at its own site. The direction that separates good attempts from bad ones does not exist until the model has tried. Geometry is the exception. There the pre-attempt state already predicts failure, so that cell is mostly difficulty.

Reading before the question. A subtler worry. Every read so far came after the model was asked to assess itself. Maybe the question creates the signal rather than reveals it. So we read the state at the last token of the answer, before any confidence question exists. On MMLU-Pro, correctness is readable there at 0.69. From the question alone it is 0.57, so the model cannot see failure coming. Asking the model to self-assess sharpens the signal to 0.77.

Transfer across domains. Table 1 trains a probe on one domain and tests it on each of the others. On Mistral it keeps about 90% of its lift above chance (0.69 off the diagonal against 0.71 on it). The full four-model matrix showed the same pattern. The one exception is Gemma-4. There, probes trained on geometry fail elsewhere (0.31 to 0.39), while probes trained elsewhere work on geometry (about 0.70). A probe trained on the one generative task can learn features specific to it.

4 Whether self-report depends on the residual signal

Intervention. A model could carry a perfect internal record of its errors and never consult it during self-report. The only way to find out is to change the record and watch the number. At the confidence token, after the answer is written so the answer cannot change, we scale the model's position along the correctness direction by a gain g. Gain 1 leaves the model untouched, gain 0 erases the per-attempt information, and gains 2 and 4 exaggerate it. We run this on Mistral and MATH, 150 held-out records, with a random direction of the same size as a control. Table 2 shows what the model wrote at each setting.

Result on Mistral. Erase the direction and self-report loses its information: AUROC falls from 0.84 to 0.56, a change of -0.27 (95% bootstrap interval -0.42 to -0.12). Erase a random direction and nothing happens (-0.00, interval -0.03 to $+0.03$). This is not generic damage. The output stays well formed, and confidence on wrong answers goes *up* to match right answers, so the model becomes uniformly sure rather than confused. Exaggerate the direction and confidence on wrong answers falls while right answers hold; expected calibration error halves, $0.34 \rightarrow 0.19$. Two checks say the effect is specific. The same intervention 30%

Table 2: Mistral on MATH, 150 held-out records. The correctness direction scaled by a gain at the confidence token, after the answer is written. Bold marks the erased and quadrupled settings; unparseable means the output no longer parsed as a confidence.

gain on the correctness direction	**untouched** 1	**erased** 0	**doubled** 2	**quadrupled** 4
mean confidence, wrong / right answers	84 / 96	**97 / 98**	70 / 94	**54 / 88**
AUROC of self-report	0.84	**0.56**	0.85	0.82
same, with a random direction	0.84	0.83	0.84	unparseable

of the way through the network does nothing, and a direction learned on MMLU-Pro, where surface features cannot explain the probe, still controls the MATH report (0.88 $\to$ 0.71).

Result on GLM. GLM has the strongest signal in the study: within a question on MATH, the probe reads 0.87 while the model's own P(True) reads 0.53. Yet erasing the direction leaves self-report where it was, 0.76 before and after. GLM represents its errors more sharply than Mistral and reports them less. This is our cleanest case of represented but not reported, and it makes the causal result a fact about particular models, not models in general.

5 The Jacobian lens

A label-free check. A trained probe might have overfit to our data. So we add an instrument that uses no correctness labels. The Jacobian lens [3] is fit on generic text and reads an activation for what it pushes the model to say next; we score each state by how far it leans toward "wrong" over "correct." On dense models the lens matches the probe: on Mistral it reads 0.82 on MATH and 0.75 on MMLU-Pro, against the probe's 0.83 and 0.75, and reads 0.5 at the input layer, as it should. Two instruments built from different evidence find the same direction, and because the lens reads through the model's own output pathway, that direction sits where it could influence what the model says.

Where the lens fails. The lens is a fixed dictionary from hidden-state directions to words, built once from generic text. On a dense model one dictionary is enough, because every input takes the same path to the output. GLM routes each input through a different subset of experts, and Qwen3.6 mixes two kinds of attention layers, so there is no single path and the lens averages many into a blur. The numbers show it: on GLM the lens's entries for "correct" and "wrong" point almost the same way, cosine 0.96, against 0.40 on a dense model, and it scores 0.31 there and 0.43 on Qwen3.6, while the probe still works on both. Swap Qwen3.6 for Qwen2.5-14B, same family but dense, and the lens jumps to 0.88. The signal is there in every model. A label-free reader works only where the architecture gives it one path.

6 What this means

Practical use. Without an exact checker, the probe is the best per-attempt readout we found. Keeping the attempts it scores highest raises accuracy far more than keeping the ones the model is most confident in, and it works where the model's confidence is useless.

Limitations. A probe could be reading properties of failed output rather than a self-assessment; the surface baseline bounds this without closing it, most loosely on MATH, geometry (Appendix C), and GLM. Every number is a single seed. The before-asking test runs only on MMLU-Pro; on MATH the question alone already predicts correctness at 0.79. The causal result holds on Mistral and fails on GLM; Gemma-4 leaves too few failures to test, and the Qwen3.6 run did not finish in time. Whether preference tuning creates the gap is open; a base model would answer it.

Takeaway. These models carry information about their own errors that their self-report does not, and on one model self-report depends on it.

References

[1] Amos Azaria and Tom Mitchell. The internal state of an LLM knows when it's lying. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023.

[2] Collin Burns, Haotian Ye, Dan Klein, and Jacob Steinhardt. Discovering latent knowledge in language models without supervision. In *International Conference on Learning Representations (ICLR)*, 2023.

[3] Wes Gurnee, Nicholas Sofroniew, Adam Pearce, Mateusz Piotrowski, Isaac Kauvar, Runjin Chen, Anna Soligo, Paul Bogdan, Euan Ong, Rowan Wang, T. Ben Thompson, David Abrahams, Subhash Kantamneni, Emmanuel Ameisen, Joshua Batson, and Jack Lindsey. Verbalizable representations form a global workspace in language models. *Transformer Circuits Thread*, 2026. `https://transformer-circuits.pub/2026/workspace/index.html`; code: `https://github.com/anthropics/jacobian-lens`.

[4] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, et al. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*, 2022.

[5] Samuel Marks and Max Tegmark. The geometry of truth: Emergent linear structure in large language model representations of true/false datasets. In *Conference on Language Modeling (COLM)*, 2024.

[6] Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Alexander Pan, Xuwang Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, et al. Representation engineering: A top-down approach to AI transparency. *arXiv preprint arXiv:2310.01405*, 2023.

A The elicitation protocol, and one record end to end

This appendix gives the exact prompts behind every record, then one complete record from the Mistral-Small-24B × MATH cell, summarized first and then given in full.

> **One record, Mistral-Small-24B × MATH.** *What integer n satisfies $0 \le n < 18$ and $n \equiv -11213141 \pmod{18}$?* The model sets the problem up correctly and gets $18 \times 622953 = 11213154$ right, then writes $r = -11213141 + 11213154 = 3$ and answers 3. That subtraction gives 13, which is the gold answer. Stated confidence: 90 before the attempt, 95 after it. Other readouts of the same attempt: P(True), the probability the model assigns to "True" when asked whether its answer is correct, 0.32; mean answer log-probability -0.05.

A single arithmetic slip inside a fluent derivation. Having just made the error, the model revises its stated confidence upward, while P(True) on the same transcript assigns only 0.32 probability to correctness. The rest of this appendix gives the prompts and the full transcript.

A.1 Prompts

Each record is four forward passes: the three turns of Section 2 plus one read taken beforehand, at the last prompt token of the bare question, before any elicitation. The confidence turns run with thinking off so the read lands on a clean one-line reply; the attempt turn runs with thinking on for hybrid reasoners. The fixed `Confidence: N` format is what makes the read site content-neutral: the digit token has identical local context on every record, so layer-0 activations carry no signal by construction.

System prompt (MATH).

```
Solve the problem step by step. Put your final answer in \boxed{}.
```

Turn 1, prospective confidence. The problem is followed by:

```
Before answering, estimate how likely you are to answer this question
correctly. Reply with EXACTLY one line and nothing else:
```

```
Confidence: N
where N is an integer from 0 (certain to be wrong) to 100 (certain to be
correct). Do NOT answer yet.
```

Turn 2, the attempt. The turn-1 reply is left in context as the assistant message, then:

```
Now give your final answer inside \boxed{}.
```

Turn 3, retrospective confidence. The attempt is left in context, then:

```
Now assess the answer you just gave. On a scale of 0 to 100, how confident
are you that it is correct? Reply with exactly one line and nothing else:
Confidence: N
```

P(True) baseline. A separate teacher-forced branch off turn 2, no generation. Next-token mass on `True` versus `False`:

```
Consider your final answer above. Is it correct? Reply with exactly one word:
True or False.
```

Grading is external and is never shown to the model, so the turn-3 revision is blind self-assessment.

A.2 One record, verbatim

Record `math_test_number_theory_753`, sample 2 of 5.

Problem. *What integer n satisfies $0 \le n < 18$ and $n \equiv -11213141 \pmod{18}$?*

Turn 1 reply. `Confidence: 90`

Turn 2 reply (complete).

> First, we need to find the remainder when -11213141 is divided by 18.
> 1. Calculate $-11213141 \bmod 18$:
>
> $$-11213141 \div 18 \approx -622952.2778$$
>
> The integer part of the division is -622953.
>
> $$-11213141 = 18 \times (-622953) + r$$
>
> where r is the remainder.
> 2. Calculate the remainder r:
>
> $$r = -11213141 - 18 \times (-622953)$$
>
> $$r = -11213141 + 11213154$$
>
> $$r = 3$$
>
> Therefore,
>
> $$-11213141 \equiv 3 \pmod{18}$$
>
> So, the integer n that satisfies $0 \le n < 18$ and $n \equiv -11213141 \pmod{18}$ is 3.

Turn 3 reply. `Confidence: 95`

What happened. The method is right and every intermediate is right: $18 \times 622953 = 11213154$ checks out, and the model has written the correct subtraction on the line above its answer. It then evaluates $-11213141 + 11213154$ as 3 rather than 13. Gold is 13. Nothing in the presentation marks the slip. The derivation is well formed, correctly structured, and confidently laid out, which is precisely why stated confidence rises from 90 to 95 across an attempt that failed.

Readouts of this attempt.

readout	value
stated confidence, prospective (turn 1)	90
stated confidence, retrospective (turn 3)	95
P(True)	0.32
mean answer log-probability	−0.05
graded outcome	**incorrect**

This is one record, chosen to illustrate the failure mode rather than to carry evidential weight; representative cell-level numbers are in Table 3, Appendix D.

B Calibration beyond ECE, and cell-level multiplicity

ECE depends on binning and on sample composition, so we report two binning-free proper scoring rules alongside it. On Mistral × MATH (750 records, accuracy 0.648), the model's retrospective stated confidence gives Brier 0.262 and NLL 1.470 against a 10-bin ECE of 0.271. P(True) on the same records gives Brier 0.386: better ordered than stated confidence, worse calibrated in absolute terms. The steering results in Table 2 are reported as ECE because the steered runs stored per-gain aggregates rather than per-record confidences; recomputing Brier and NLL at each gain requires re-running the intervention and is the first thing we would add.

The cell-level bootstrap counts in Table 3 are uncorrected. With 16 cells, some individually significant results are expected under the null, so the cell counts should be read as descriptive. The aggregate pattern is the claim that carries weight: the effect is positive in 12 of 16 cells with one tie, significantly positive in 10, and never significantly negative. A hierarchical model pooling across cells with model and domain as random effects is the right test and is not yet run.

C Alternative explanations for the probe signal

A probe could key on properties of failed derivations rather than on a dedicated metacognitive variable: low token probability, hedging language, or, on MATH, an early-layer channel next to the answer text. Our surface-features baseline (answer length, digit count, line count, word count, brace count) bounds this without closing it, and the bound is domain dependent: adding the probe to the surface features raises AUROC by +0.21 on MMLU-Pro but only +0.07 on MATH and −0.11 on geometry, where a construction that fails to compile is visibly malformed. Two results argue against a pure surface account on the domains where the signal is strong. A direction fitted on MMLU-Pro, where surface reaches only 0.54 against a probe of 0.75, still governs MATH's self-report under ablation (0.880 → 0.709). And the pre-attempt read, taken where the activation is byte-identical across a question's attempts, cannot recover the post-attempt direction on MMLU-Pro (0.490, chance), which a surface-of-the-question account would predict it could. A linear probe at one token also undercounts nonlinear or multi-token signal, so the reported numbers are a floor.

D Per-cell numbers and specificity controls

Table 3: Post-attempt correctness discrimination (AUROC). Left: self-report vs. the probe in representative cells, and the same comparison holding the question fixed ("within"), where the model's yardstick is P(True) across the five attempts at one question; overall the probe wins 12 of 16 with one tie, mean +0.09, with a paired bootstrap significantly positive in 10 of 16 and never significantly negative, uncorrected at the cell level (Appendix B). Right: Mistral specificity controls. "pre" is the last prompt token before the attempt; "post" is after the attempt; "resid." removes the pre score; "post→pre" applies that fitted post readout at the pre-attempt site, where only difficulty is visible.

model × domain	self-report	probe	within: P(True)	within: probe
Gemma-4 × MATH	0.57	**0.78**	–	–
Qwen3.6 × MMLU-Pro	0.67	**0.84**	0.47	**0.74**
GLM × MMLU-Pro	0.67	**0.85**	0.59	**0.73**
Mistral × MATH	0.79	**0.83**	0.59	**0.72**

Mistral	pre	post	resid.	post→pre
MMLU-Pro	0.568	0.754	0.751	0.490
MATH	0.792	0.834	0.701	0.624
GPQA	0.593	0.572	0.544	0.479
GeoGen	0.770	0.676	0.505	0.648

E Checkpoints

All models are public Hugging Face releases, run in bfloat16 without quantization, greedy decoding: `mistralai/Mistral-Small-24B-Instruct-2501`, `Qwen/Qwen3.6-27B`, `zai-org/GLM-4.7-Flash`, `google/gemma-4-26B-A4B-it`; the dense within-family control for the lens is `Qwen/Qwen2.5-14B-Instruct`.